

CHƯƠNG 3: THIẾT KẾ CƠ SỞ DỮ LIỆU VÀ KIẾN TRÚC CHI TIẾT

Chương 2 đã trình bày mô hình phân tích ở mức nghiệp vụ, trong đó hệ thống được khái quát hóa thành các thực thể Sinh viên, Admin, FAQ, Message và Feedback cùng các mối quan hệ giữa chúng. Đây là mô hình phù hợp để trình bày logic nghiệp vụ, làm cơ sở trao đổi với giảng viên hướng dẫn và thống nhất phạm vi đề tài trong nhóm, song khi bắt tay vào cài đặt, nhóm nhận thấy mô hình khái niệm này chưa đủ để vận hành một hệ thống chatbot có khả năng trả lời tự động dựa trên dữ liệu học vụ thật, quản lý phiên hội thoại và xử lý các yêu cầu hỗ trợ khi bot không đủ dữ liệu để trả lời. Chương 3 trình bày quá trình cụ thể hóa mô hình đó thành schema cơ sở dữ liệu MySQL, các endpoint xử lý nghiệp vụ và giao diện người dùng có thể chạy được trên môi trường triển khai thực tế của nhóm, gồm PHP, MySQL và một service Python riêng cho chức năng đọc văn bản.

Cần nói rõ một điểm trước khi đi vào chi tiết: việc schema triển khai có nhiều bảng hơn so với năm thực thể đã trình bày ở Chương 2 không có nghĩa là nhóm thay đổi phạm vi hay mục tiêu của đề tài. Nhóm chỉ tách các phần dữ liệu mà mô hình khái niệm gộp chung thành nhiều bảng chuyên biệt hơn để phục vụ đúng yêu cầu kỹ thuật: theo dõi nguồn tài liệu, kiểm duyệt nội dung trước khi đưa vào câu trả lời, chia nhỏ nội dung dài thành các đoạn để tìm kiếm chính xác hơn, và ghi lại lịch sử truy vấn để tiện kiểm tra sau này. Bảng users kết hợp roles vẫn đóng đúng vai trò quản lý tài khoản Sinh viên và Admin đã mô tả ở Chương 1; nhóm bảng knowledge_articles và knowledge_chunks vẫn đóng vai trò của FAQ nhưng ở dạng có thể tra cứu theo ngữ nghĩa; chat_messages và chat_feedback vẫn đóng đúng vai trò của Message và Feedback. Phần mở rộng đáng kể nhất nằm ở nhóm bảng ticket — phần xử lý khi chatbot không trả lời được — vốn là một nhánh mở rộng tự nhiên của Use Case "Báo cáo câu trả lời không hợp lệ" đã có ở Chương 2.

3.1. Chi tiết hóa schema triển khai thực tế (MySQL)

Toàn bộ schema được cài đặt trên MySQL phiên bản 8.0 trở lên, sử dụng engine InnoDB cho các bảng nghiệp vụ để tận dụng khả năng ràng buộc khóa ngoại và giao dịch. Việc tạo cơ sở dữ liệu được thực hiện thông qua file database/do_an_udpm_database_complete.sql, trong đó cơ sở dữ liệu do_an_udpm được tạo với bộ ký tự utf8mb4 nhằm hỗ trợ đầy đủ tiếng Việt có dấu, múi giờ hệ thống được đặt về +07:00 cho phù hợp với dữ liệu học vụ trong nước. Bên cạnh file schema chính, nhóm còn có database_full_export.sql là bản xuất tổng hợp toàn bộ cấu trúc và

dữ liệu tại một thời điểm, dùng để đối chiếu hoặc khôi phục khi cần thiết; hai file `fix_users.sql` và `seed_student.sql` được dùng riêng cho việc chèn dữ liệu tài khoản và dữ liệu học vụ mẫu phục vụ kiểm thử.

3.1.1. Kiểu dữ liệu, ràng buộc và index chi tiết từng bảng

a) Nguyên tắc thiết kế chung

Trước khi đi vào từng bảng cụ thể, cần trình bày ba nguyên tắc mà nhóm thống nhất áp dụng xuyên suốt schema, vì đây là các quyết định lặp lại ở hầu hết các bảng và giải thích được vì sao schema được tổ chức như hiện tại thay vì theo một cách khác.

Nguyên tắc thứ nhất là dùng số nguyên không dấu cho mọi khóa chính và khóa ngoại có khả năng bị tham chiếu chéo giữa nhiều bảng. Lựa chọn này giúp các thao tác kết nối giữa những bảng có quan hệ chặt như `enrollments`, `subjects`, `course_sections` và `semesters` có chi phí xử lý thấp hơn so với dùng chuỗi làm khóa, đồng thời tiết kiệm không gian lưu trữ hơn so với việc dùng định danh dạng văn bản cho những bảng vốn không cần định danh phân tán qua nhiều hệ thống.

Nguyên tắc thứ hai là dùng kiểu liệt kê cho các trường có tập giá trị nghiệp vụ đóng, chẳng hạn trạng thái tài khoản, trạng thái ticket hay loại đánh giá của sinh viên đối với câu trả lời của bot. Việc này chuyển một phần công việc kiểm tra tính hợp lệ của dữ liệu xuống tầng cơ sở dữ liệu thay vì chỉ dựa vào kiểm tra ở tầng ứng dụng, nhờ đó giảm khả năng một bản ghi rơi vào trạng thái không có ý nghĩa nghiệp vụ do sai sót khi lập trình.

Nguyên tắc thứ ba liên quan trực tiếp đến chức năng tìm kiếm của chatbot: những nội dung dài và cần tra cứu theo từ khóa như câu trả lời tri thức, mô tả ticket, nội dung nguồn tài liệu hay thông báo đều được đánh chỉ mục toàn văn thay vì chỉ dựa vào phép so khớp chuỗi thông thường. Đây là quyết định gắn liền với chức năng cốt lõi của hệ thống, bởi chatbot cần tìm nhanh đoạn văn bản phù hợp với câu hỏi của sinh viên, và chỉ mục toàn văn của MySQL cho phép thực hiện việc này ngay ở tầng cơ sở dữ liệu mà không cần tải toàn bộ dữ liệu tri thức lên tầng ứng dụng để so khớp thủ công.

b) Phân nhóm các bảng theo chức năng

Toàn bộ schema có thể chia thành tám nhóm theo chức năng nghiệp vụ như tóm tắt ở Bảng 3.1. Cách phân nhóm này không tồn tại như một schema riêng biệt trong cơ sở dữ liệu mà là cách nhóm tổ chức để trình bày và để dễ kiểm soát trong quá trình phát triển.

Nhóm chức năng	Các bảng chính	Vai trò dữ liệu
Xác thực và tổ chức	roles, users, faculties, programs, student_profiles	Tài khoản, vai trò, đơn vị đào tạo và hồ
Học vụ	academic_years, semesters, subjects, course_sections, class_schedule_sessions, enrollments, exam_schedules, grades	Năm học, học kỳ, môn học, lớp học phần học/thi và điểm
Học phí	tuition_invoices, tuition_invoice_items, tuition_payments, tuition_extension_requests	Hóa đơn, chi tiết khoản thu, giao dịch và gia hạn
Thông báo	announcements, announcement_audiences, academic_deadlines	Tin tức, đối tượng nhận thông báo và hạn vụ
Kho tri thức	knowledge_sources, knowledge_articles, knowledge_chunks, faq	Nguồn tài liệu, nội dung tri thức đã kiến dữ liệu FAQ cũ
Chatbot và RAG	chat_sessions, chat_messages, rag_retrieval_logs, chat_feedback, unanswered_questions	Phiên chat, tin nhắn, nhật ký truy xuất và của sinh viên
Hỗ trợ	tickets, ticket_messages, ticket_attachments	Yêu cầu hỗ trợ, trao đổi hai chiều và tệp
Vận hành	system_settings, uploaded_documents, audit_logs, api_usage_logs	Cấu hình hệ thống, nhập tài liệu và nhật

Bảng 3.1. Phân nhóm các bảng trong schema triển khai

Do số lượng bảng khá lớn, phần dưới đây không trình bày chi tiết toàn bộ mà tập trung vào bốn nhóm có ảnh hưởng trực tiếp đến khả năng vận hành của chatbot: nhóm tài khoản, nhóm kho tri thức, nhóm hội thoại/RAG và nhóm ticket. Các nhóm còn lại được thiết kế theo cùng những nguyên tắc đã nêu ở phần a) nên không lặp lại chi tiết.

c) Nhóm tài khoản, phân quyền và hồ sơ sinh viên

Việc tách vai trò người dùng ra một bảng roles riêng thay vì lưu trực tiếp một chuỗi quyền hạn trong bảng users là quyết định nhằm chuẩn hóa dữ liệu và tránh sai sót khi ghi nhận quyền. Bốn vai trò hiện có trong hệ thống là student, admin, staff và knowledge_reviewer. Vai trò cuối cùng phản ánh một nhu cầu phát sinh trong quá trình phát triển: người kiểm duyệt nội dung tri thức trước khi đưa vào chatbot không nhất thiết phải có toàn bộ quyền quản trị hệ thống như Admin, nên được tách thành một vai trò riêng dù chưa được mô tả tường minh trong sơ đồ Use Case ở Chương 2.

Bảng users giữ vai trò tài khoản trung tâm, lưu tên đăng nhập, email, mật khẩu đã băm và không lưu văn bản thô, cùng vai trò và trạng thái tài khoản. Việc tách student_profiles ra khỏi users, thay vì gộp chung thông tin sinh viên vào bảng tài khoản, xuất phát từ thực tế không phải tài khoản nào cũng là sinh viên: Admin và Staff không có mã số sinh viên, khóa học hay chương

trình đào tạo. Quan hệ một - một giữa hai bảng được ràng buộc bằng khóa ngoại duy nhất; khi một tài khoản bị xóa, hồ sơ sinh viên tương ứng cũng được xóa theo để tránh còn sót lại bản ghi không có chủ. Ngược lại, quan hệ giữa hồ sơ sinh viên và chương trình đào tạo được thiết kế theo hướng giữ lại hồ sơ khi chương trình đào tạo bị xóa khỏi hệ thống, chỉ mất liên kết chứ không mất dữ liệu sinh viên, nhằm tránh việc một thay đổi ở bảng danh mục làm ảnh hưởng đến dữ liệu cá nhân của sinh viên.

d) Nhóm kho tri thức

Đây là phần mở rộng đáng kể nhất so với mô hình FAQ đơn giản ở Chương 2, và cũng là phần quyết định chất lượng câu trả lời của chatbot. Thay vì lưu trực tiếp cặp câu hỏi - câu trả lời như một bảng FAQ thông thường, nhóm tổ chức kho tri thức theo ba lớp: `knowledge_sources` lưu nguồn gốc của thông tin gồm loại văn bản, đơn vị ban hành, số hiệu và trạng thái còn hiệu lực; `knowledge_articles` lưu nội dung câu trả lời đã được biên tập kèm theo phạm vi áp dụng và trạng thái kiểm duyệt; `knowledge_chunks` chia nội dung dài thành các đoạn nhỏ hơn để phục vụ tìm kiếm.

Lý do cho cách tổ chức ba lớp này xuất phát từ một bài toán thực tế mà một hệ thống FAQ đơn giản không giải quyết được: quy chế học vụ, lịch thi hay chính sách học phí không chỉ có một phiên bản duy nhất mà thay đổi theo từng học kỳ, từng khóa sinh viên, và không phải văn bản nào cũng áp dụng cho toàn trường. Nếu chỉ lưu một bảng câu hỏi - câu trả lời phẳng, hệ thống sẽ không có cách nào phân biệt được một câu trả lời đã hết hiệu lực với một câu trả lời đang áp dụng, hoặc một câu trả lời chỉ dành cho khóa tuyển sinh cũ. Việc bổ sung các trường về thời hạn hiệu lực, trạng thái kiểm duyệt và liên kết đến nguồn giúp chatbot chỉ lấy dữ liệu đã được xác minh và còn hiệu lực khi trả lời, thay vì trả lời dựa trên toàn bộ dữ liệu chưa qua kiểm duyệt.

Bảng faq theo mô hình cũ vẫn được giữ lại trong schema, đóng vai trò lớp tương thích ngược với dữ liệu FAQ ban đầu của nhóm. Trong quá trình phát triển, phần mã nguồn xử lý FAQ vẫn nhận diện được cả cách đặt tên cột kiểu cũ lẫn kiểu mới, giúp nhóm không phải viết lại toàn bộ phần xử lý FAQ cũ trong lúc chuyển sang mô hình kho tri thức chuẩn hóa.

e) Nhóm hội thoại và truy xuất RAG

Một phiên trò chuyện được định danh bằng một mã UUID riêng thay vì chỉ dựa vào mã tài khoản, vì hệ thống cần hỗ trợ cả trường hợp người dùng chưa đăng nhập vẫn có thể trò chuyện với bot ở mức giới hạn. Mỗi tin nhắn, của cả người dùng lẫn của bot, được lưu lại kèm theo ý

định đã được phân loại, các thực thể được trích xuất như mã học kỳ hay mốc thời gian, và trạng thái của câu trả lời, phục vụ việc thống kê chất lượng bot về sau.

Bảng nhật ký truy xuất là nơi ghi lại toàn bộ quá trình chatbot cân nhắc trước khi trả lời: mỗi đoạn tri thức được xem xét đều được chấm điểm theo nhiều tiêu chí khác nhau, gồm mức độ khớp từ khóa, mức độ khớp ngữ nghĩa, mức độ phù hợp ngữ cảnh và độ mới của thông tin, cùng với điểm tổng hợp cuối cùng và cờ đánh dấu đoạn đó có được chọn đưa vào câu trả lời hay không. Việc ghi log chi tiết này không phục vụ người dùng cuối mà phục vụ chính nhóm phát triển: khi bot trả lời sai, nhóm có thể tra lại chính xác những đoạn tri thức nào đã được xem xét và vì sao đoạn đó được hoặc không được chọn, thay vì chỉ phỏng đoán dựa trên câu trả lời cuối cùng.

Trường hợp không có đoạn tri thức nào đạt ngưỡng điểm cần thiết, câu hỏi đó được ghi vào một bảng riêng dành cho câu hỏi chưa trả lời được, kèm bộ đếm số lần câu hỏi tương tự xuất hiện lại. Đây là cơ chế giúp Admin nhận biết những khoảng trống tri thức cần được bổ sung dựa trên tần suất câu hỏi thực tế của sinh viên, thay vì dựa trên phỏng đoán chủ quan.

f) Nhóm ticket hỗ trợ

Nhóm bảng tickets, ticket_messages và ticket_attachments hiện thực hóa phần mở rộng của Use Case Báo cáo câu trả lời không hợp lệ đã mô tả ở Chương 2, nhưng đi xa hơn một báo cáo lỗi đơn thuần: đây là một luồng trao đổi hai chiều giữa sinh viên và Admin, tương tự một hệ thống hỗ trợ thu nhỏ. Một ticket có thể được tạo từ hai nguồn, do sinh viên chủ động yêu cầu hỗ trợ hoặc do chatbot tự đề nghị khi không tìm được câu trả lời phù hợp, và mỗi ticket giữ trạng thái riêng để cả hai phía cùng theo dõi được tiến độ xử lý.

g) Tổng hợp một số ràng buộc và index tiêu biểu

Bảng	Ràng buộc / Index	Mục đích triển khai
users	Khóa duy nhất trên tên đăng nhập và email	Bảo đảm mỗi tài khoản có định danh lặp
student_profiles	Khóa ngoại một - một với users, khóa duy nhất trên mã số sinh viên	Không cho phép hai tài khoản dùng cùng hồ sơ hoặc hai hồ sơ trùng MSSV
knowledge_articles	Chỉ mục toàn văn, chỉ mục theo trạng thái kiểm duyệt và mức ưu tiên	Tìm kiếm nội dung và lọc dữ liệu đã duyệt
knowledge_chunks	Khóa duy nhất trên cặp bài viết - số thứ tự đoạn, chỉ mục toàn văn	Giữ đúng thứ tự các đoạn và hỗ trợ tìm kiếm nội dung
chat_feedback	Khóa duy nhất trên cặp tin nhắn - người dùng	Ngăn một sinh viên đánh giá trùng nhau cùng câu trả lời

Bảng	Ràng buộc / Index	Mục đích triển khai
tickets	Khóa duy nhất trên mã ticket, chỉ mục theo trạng thái và mức ưu tiên	Quản lý danh sách ticket cần xử lý theo khẩn cấp

Bảng 3.2. Một số ràng buộc và chỉ mục tiêu biểu liên quan trực tiếp đến chatbot

h) Các view hỗ trợ truy vấn an toàn

Ngoài các bảng vật lý, schema có một số view được tạo sẵn nhằm chuẩn hóa cách truy vấn dữ liệu, trong đó quan trọng nhất đối với chatbot là view chỉ lấy các bài tri thức đã qua kiểm duyệt, còn trong thời hạn hiệu lực và thuộc nguồn đang hoạt động. Việc tạo sẵn một view dùng chung cho mục đích này, thay vì để mỗi endpoint tự viết điều kiện lọc riêng, giúp bảo đảm mọi nơi trong hệ thống truy vấn dữ liệu tri thức đều đi qua cùng một bộ điều kiện kiểm duyệt, tránh trường hợp một endpoint vô tình lấy phải dữ liệu chưa được duyệt để trả lời sinh viên.

3.1.2. Đối chiếu với script triển khai thực tế

Quá trình triển khai cơ sở dữ liệu của nhóm không diễn ra trong một bước duy nhất mà được chia thành các lớp script với vai trò khác nhau, phản ánh đúng trình tự công việc thực tế khi xây dựng hệ thống.

File database/do_an_udpm_database_complete.sql là script nền tảng, đảm nhiệm việc tạo cơ sở dữ liệu, tạo toàn bộ bảng và view, sau đó nhập dữ liệu FAQ ban đầu và chuyển đổi dữ liệu này sang cấu trúc kho tri thức chuẩn hóa. Đây là bước ghi nhận rõ ràng nhất sự chuyển tiếp từ mô hình FAQ đơn giản ban đầu, vốn là điểm khởi đầu của đồ án, sang mô hình tri thức có kiểm duyệt mà nhóm phát triển thêm trong quá trình làm việc.

File database_full_export.sql là bản xuất tổng hợp toàn bộ cơ sở dữ liệu tại một thời điểm cụ thể, phục vụ mục đích sao lưu và đối chiếu, khác với script schema chính vốn mô tả quy trình dựng cơ sở dữ liệu từ đầu.

Hai file fix_users.sql và seed_student.sql phục vụ việc tạo dữ liệu mẫu để kiểm thử: file thứ nhất bổ sung tài khoản Admin và sinh viên mẫu cùng hồ sơ tương ứng, file thứ hai bổ sung dữ liệu học vụ mẫu như môn học, lớp học phần, lịch học và dữ liệu đăng ký, để các tính năng tra cứu lịch học và tra cứu điểm có dữ liệu hiển thị khi kiểm thử. Vì đây là các script bổ sung độc lập với nhau, nhóm cần chạy đúng thứ tự, tạo schema trước rồi mới chạy các script seed, đồng thời cần đối chiếu lại thông tin tài khoản và mật khẩu mẫu giữa hai file này trước khi dùng để kiểm thử đăng nhập, tránh trường hợp một script seed chạy sau ghi đè dữ liệu của script chạy trước.

Nhìn chung, nhóm không xem mô hình năm thực thể ở Chương 2 là danh sách bảng vật lý cuối cùng phải tuân theo tuyệt đối, mà xem đó là mô hình nghiệp vụ cốt lõi cần được giữ nguyên ý nghĩa trong suốt quá trình phát triển. Các bảng bổ sung sau này, gồm nguồn tri thức, đoạn văn bản, nhật ký truy xuất, ticket và cấu hình hệ thống, là kết quả của việc cụ thể hóa các yêu cầu kỹ thuật nảy sinh khi cài đặt thực tế, không làm thay đổi phạm vi và mục tiêu đề tài đã được phê duyệt ở Chương 1.

[CHÈN HÌNH 3.1: Sơ đồ ánh xạ từ các thực thể khái niệm ở Chương 2 sang các bảng triển khai trong schema MySQL]

3.2. Thiết kế API và Backend

3.2.1. Cấu trúc thư mục dự án

Về mặt tổ chức mã nguồn, hệ thống được xây dựng theo hướng phân lớp trách nhiệm giữa giao diện, endpoint xử lý nghiệp vụ và logic dùng chung, khá gần với tinh thần kiến trúc ba tầng đã trình bày ở Chương 1, nhưng chưa hoàn toàn theo mô hình MVC kinh điển với các thư mục controller và model tách biệt rõ ràng. Cụ thể, thư mục views đảm nhiệm cả vai trò hiển thị lẫn một phần xử lý nghiệp vụ ngay trong file PHP, chẳng hạn tiếp nhận form đăng nhập và kiểm tra quyền truy cập theo từng trang; thư mục api là nơi tiếp nhận các request từ JavaScript phía client và điều phối nghiệp vụ chính; thư mục core chứa các hàm dùng chung, đặc biệt là toàn bộ thao tác kết nối và truy vấn cơ sở dữ liệu; còn thư mục tts là một service độc lập viết bằng Python, tách khỏi phần lõi PHP để không phụ thuộc vào các thư viện chuyển giọng nói vốn không có sẵn cho PHP.

do_an_udpm/	
├─ api/	(các endpoint xử lý nghiệp vụ: chatbot, feedback, ticket, TTS...)
├─ core/	(hàm dùng chung, kết nối và truy vấn cơ sở dữ liệu)
├─ css/	(style riêng cho từng khu vực: đăng nhập, dashboard, admin)
├─ js/	(xử lý phía client: chatbot.js, admin.js)
├─ database/	(schema, script seed, công cụ kiểm tra dữ liệu)
├─ tts/	(service Python FastAPI cho chức năng đọc văn bản)
├─ uploads/	(tệp người dùng tải lên: avatar, tài liệu)
├─ views/	(giao diện PHP: login, dashboard, admin_dashboard)
├─ config.php	(đọc biến môi trường, cấu hình kết nối MySQL và khóa AI)
└─ .env	(thông tin cấu hình nhạy cảm, không đưa lên mã nguồn công khai)

Thành phần	Vai trò triển khai
views/login.php	Nhận thông tin đăng nhập, xác thực và điều hướng theo vai trò tài khoản
views/dashboard.php	Giao diện và nghiệp vụ phía Sinh viên: dashboard, chatbot, lịch sử, ticket
views/admin_dashboard.php	Giao diện quản trị: kiểm duyệt tri thức, quản lý người dùng, xử lý ticket
api/chatbot.php	Phân loại câu hỏi, truy vấn dữ liệu, truy xuất RAG và trả câu trả lời

Thành phần	Vai trò triển khai
api/*.php (còn lại)	Các endpoint cho đánh giá câu trả lời, ticket, hội thoại, TTS và avatar
core/faq_helpers.php	Kết nối cơ sở dữ liệu, chuẩn hóa dữ liệu, truy vấn dùng chung cho toàn hệ thống
config.php	Đọc cấu hình từ .env, thiết lập kết nối MySQL và khóa dịch vụ AI
js/chatbot.js	Gửi câu hỏi, hiển thị câu trả lời, lưu lịch sử cục bộ, gọi TTS và gửi đánh giá
tts/server.py	Service FastAPI tạo file âm thanh từ văn bản

Bảng 3.3. Vai trò của các thành phần chính trong kiến trúc backend

Việc gom toàn bộ thao tác kết nối cơ sở dữ liệu và các hàm xử lý dùng chung vào một file duy nhất trong thư mục core là một quyết định có chủ đích: nếu để mỗi endpoint trong api tự viết kết nối và câu lệnh SQL riêng, khi cần thay đổi cấu trúc bảng hoặc sửa lỗi truy vấn, nhóm sẽ phải sửa ở nhiều nơi và dễ bỏ sót. Toàn bộ câu lệnh SQL trong các hàm này sử dụng prepared statement với tham số truyền riêng, không nối trực tiếp giá trị người dùng nhập vào chuỗi truy vấn, đây là biện pháp cơ bản để hạn chế lỗi SQL Injection.

Về thay đổi cơ chế xác thực so với thiết kế ban đầu

Ở giai đoạn phân tích và thiết kế tại Chương 1 và Chương 2, module đăng nhập được mô hình hóa theo hướng dùng JWT để xác thực phiên làm việc. Trong quá trình cài đặt thực tế, do toàn bộ giao diện được viết bằng PHP thuần và nhiều trang cùng chia sẻ trạng thái đăng nhập, nhóm quyết định chuyển sang dùng PHP Session kết hợp cơ chế xoay vòng session ID sau khi xác thực thành công, thay vì duy trì một token JWT riêng ở phía client.

Đây là điều chỉnh ở tầng hiện thực, không làm thay đổi yêu cầu nghiệp vụ về đăng nhập và phân quyền đã đặt ra: hệ thống vẫn kiểm tra tài khoản, xác minh mật khẩu đã bấm, kiểm tra trạng thái tài khoản trước khi cấp quyền truy cập, và mỗi trang vẫn kiểm tra đúng vai trò được phép truy cập, trang quản trị chỉ chấp nhận các vai trò Admin, Staff và người kiểm duyệt tri thức, trong khi trang dashboard sinh viên chỉ chấp nhận vai trò Sinh viên. Việc xoay vòng session ID sau khi đăng nhập thành công là biện pháp giảm nguy cơ tấn công cố định phiên. Do dự án hiện chưa có số liệu đo đạc để so sánh hiệu năng giữa hai phương án, báo cáo không khẳng định PHP Session vượt trội hơn JWT về mặt kỹ thuật; lý do chính cho quyết định này là mức độ phù hợp với kiến trúc PHP thuần đang sử dụng, giúp việc quản lý trạng thái đăng nhập giữa các trang được thực hiện tập trung và đơn giản hơn.

3.2.2. Thiết kế các Endpoint API cốt lõi

Toàn bộ endpoint nghiệp vụ được đặt trong thư mục api, phần lớn nhận và trả dữ liệu dạng JSON, riêng endpoint chuyển văn bản thành giọng nói trả về dữ liệu âm thanh. Bảng 3.4 tổng hợp các endpoint chính đang có trong hệ thống.

Endpoint	Chức năng chính	Ghi chú quyền truy cập
api/chatbot.php	Nhận câu hỏi, phân loại ý định, truy vấn dữ liệu hoặc truy xuất RAG, trả câu trả lời	Cho phép cả người dùng chưa đăng nhập ở mức giới hạn
api/chat_feedback.php	Ghi nhận đánh giá Hữu ích / Báo lỗi cho một câu trả lời	Yêu cầu đã đăng nhập
api/report_bot.php	Ghi nhận câu trả lời sai và tự động tạo ticket kèm ngữ cảnh	Gọi từ giao diện chatbot
api/create_ticket.php	Tạo ticket hỗ trợ mới theo yêu cầu của sinh viên	Yêu cầu đã đăng nhập
api/get_student_chat.php	Lấy nội dung trao đổi của một ticket	Sinh viên chỉ xem ticket của mình
api/get_ticket_chat.php	Lấy nội dung trao đổi ticket theo vai trò truy cập	Admin/Staff xem theo phân quyền
api/student_reply.php	Sinh viên gửi phản hồi tiếp theo trong một ticket đang mở	Từ chối nếu ticket đã đóng
api/text_to_speech.php	Chuyển tiếp yêu cầu đọc văn bản đến service TTS	Endpoint trung gian nội bộ
api/upload_avatar.php	Kiểm tra và lưu ảnh đại diện người dùng	Yêu cầu đã đăng nhập

Bảng 3.4. Danh sách endpoint API cốt lõi

Luồng xử lý của endpoint chatbot

Endpoint xử lý hội thoại là thành phần trung tâm của module Bot Chat. Khi nhận được câu hỏi từ người dùng, hệ thống thực hiện phân loại ý định của câu hỏi trước khi quyết định hướng xử lý tiếp theo, thay vì coi mọi câu hỏi đều thuộc dạng tra cứu tri thức chung. Các nhóm ý định hiện được phân loại gồm: yêu cầu tạo ticket hỗ trợ, yêu cầu điều hướng đến một cổng thông tin khác, câu hỏi liên quan đến dữ liệu cá nhân của sinh viên như điểm, lịch học, lịch thi hoặc học phí, câu hỏi về thông báo hoặc hạn chót, hội thoại xã giao thông thường, và câu hỏi kiến thức học vụ chung.

Cách phân nhánh này xuất phát từ nhận định rằng không phải mọi câu hỏi đều nên được xử lý bằng cùng một cơ chế tìm kiếm tri thức. Với nhóm câu hỏi liên quan đến dữ liệu cá nhân, chẳng hạn hỏi điểm học kỳ hiện tại, hệ thống không đưa những dữ liệu này vào ngữ cảnh gửi cho mô hình AI mà truy vấn trực tiếp từ các bảng nghiệp vụ liên quan đến đăng ký học phần, điểm số, lịch học và hóa đơn học phí, rồi trả lời trực tiếp từ kết quả truy vấn đó. Cách xử lý này giúp tránh rủi ro mô hình AI diễn giải sai hoặc suy đoán thêm khi xử lý dữ liệu cá nhân của sinh

viên, đồng thời bảo đảm câu trả lời cho các câu hỏi loại này luôn khớp chính xác với dữ liệu thật trong hệ thống thay vì phụ thuộc vào khả năng suy luận của mô hình ngôn ngữ.

Với nhóm câu hỏi kiến thức học vụ chung, dạng câu hỏi mà FAQ truyền thống hướng tới, hệ thống thực hiện truy xuất theo cơ chế sinh câu trả lời có tăng cường truy xuất: tìm các đoạn tri thức phù hợp nhất trong view dữ liệu đã kiểm duyệt dựa trên kết hợp nhiều tiêu chí điểm số, chỉ giữ lại những đoạn đạt một ngưỡng tin cậy tối thiểu có thể cấu hình được mà không cần sửa mã nguồn, rồi mới đưa các đoạn đã qua sàng lọc vào ngữ cảnh gửi cho mô hình AI để sinh câu trả lời. Nếu không có đoạn tri thức nào đạt ngưỡng, hệ thống không cố đoán một câu trả lời mà ghi nhận câu hỏi đó vào danh sách câu hỏi chưa trả lời được và đề nghị sinh viên tạo ticket để được hỗ trợ trực tiếp, đây là điểm nối trực tiếp với module ticket đã trình bày ở mục 3.1.1.

Mỗi lượt hỏi đáp, bất kể thuộc nhánh xử lý nào, đều được ghi lại đầy đủ để phục vụ tính năng xem lịch sử hội thoại và làm cơ sở đối chiếu khi có phản hồi hoặc báo lỗi từ sinh viên.

Cơ chế phản hồi và tạo ticket

Sau mỗi câu trả lời của bot, sinh viên có thể đánh giá Hữu ích hoặc Báo lỗi ngay trên giao diện; lựa chọn này được ghi nhận với ràng buộc không cho phép một sinh viên đánh giá trùng nhiều lần cho cùng một câu trả lời. Trường hợp sinh viên xác nhận câu trả lời sai và cần được hỗ trợ thêm, hệ thống ghi nhận phản hồi tiêu cực đồng thời tự động khởi tạo một ticket kèm theo nội dung câu hỏi và câu trả lời gốc, giúp Admin có đầy đủ ngữ cảnh khi xử lý mà không cần sinh viên phải mô tả lại từ đầu.

Module chuyển văn bản thành giọng nói

Chức năng đọc câu trả lời được tách thành một service riêng viết bằng Python, độc lập với phần lõi PHP. Lựa chọn tách riêng này giúp việc xử lý giọng nói, vốn phụ thuộc vào thư viện chuyển đổi văn bản - giọng nói không có sẵn cho PHP, không làm phức tạp thêm mã nguồn PHP chính, đồng thời cho phép service này được khởi động, dừng hoặc thay thế độc lập với phần còn lại của hệ thống. Phía PHP đóng vai trò trung gian: nhận yêu cầu từ giao diện, chuyển tiếp đến service Python, rồi trả kết quả âm thanh về cho trình duyệt, thay vì để JavaScript gọi thẳng đến service Python. Cách làm này giúp địa chỉ nội bộ của service TTS không bị lộ ra phía client. Trường hợp service Python gặp sự cố, giao diện tự động chuyển sang dùng chức năng đọc văn bản có sẵn của trình duyệt như một phương án dự phòng, bảo đảm tính năng nghe câu trả lời không hoàn toàn gián đoạn khi một thành phần phụ trợ gặp lỗi.

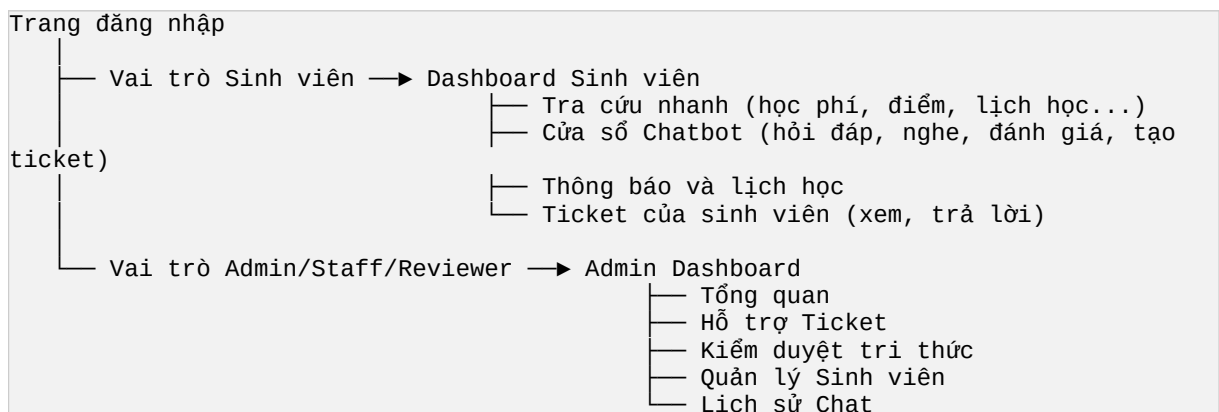
[CHÈN HÌNH 3.2: Sơ đồ tuần tự triển khai luồng xử lý câu hỏi từ giao diện đến cơ sở dữ liệu và mô hình AI]

3.3. Thiết kế Frontend và Giao diện người dùng (UI/UX)

Phần giao diện của hệ thống được xây dựng trực tiếp bằng PHP kết hợp CSS và JavaScript thuần, không sử dụng framework frontend riêng. Ba trang chính, gồm đăng nhập, dashboard sinh viên và dashboard quản trị, được tách CSS riêng theo từng khu vực, giúp việc chỉnh sửa giao diện của một vai trò người dùng không ảnh hưởng đến giao diện của vai trò còn lại.

Nguyên tắc thiết kế xuyên suốt là gom các thao tác tra cứu thường xuyên của sinh viên vào cùng một không gian làm việc thay vì phân tán ra nhiều trang riêng biệt. Những liên kết đến các hệ thống khác của trường và những chức năng cần trao đổi nhiều bước như chatbot hay ticket đều được đặt truy cập ngay từ màn hình chính, hiển thị dưới dạng cửa sổ nổi để sinh viên không phải rời khỏi ngữ cảnh đang thao tác.

3.3.1. Sơ đồ điều hướng (Navigation Map)



Việc chuyển hướng sau đăng nhập được thực hiện dựa trên vai trò của tài khoản: sinh viên được đưa đến dashboard sinh viên, trong khi Admin, Staff và người kiểm duyệt tri thức đều được đưa đến cùng một trang Admin Dashboard nhưng với các tab thao tác có thể khác nhau tùy quyền hạn. Trong Admin Dashboard, việc chuyển giữa các khu vực chức năng được thực hiện bằng tham số trên đường dẫn thay vì tải lại toàn bộ ứng dụng, giữ cho trải nghiệm quản trị được liền mạch.

3.3.2. Mockup các màn hình chính

a) Màn hình đăng nhập

Màn hình đăng nhập được chia thành hai khu vực: bên trái trình bày thông báo và tin tức của trường, bên phải là biểu mẫu đăng nhập gồm ô tài khoản, ô mật khẩu có nút hiện/ẩn và nút xác nhận. Thông báo lỗi được hiển thị ngay trên trang khi tài khoản không tồn tại, tài khoản chưa

được kích hoạt hoặc mật khẩu không khớp, giúp sinh viên biết ngay nguyên nhân thay vì chỉ nhận một thông báo lỗi chung chung.

[CHÈN HÌNH 3.5: Ảnh giao diện màn hình đăng nhập]

b) Dashboard Sinh viên

Dashboard sinh viên trình bày thông tin cá nhân, các thẻ thống kê và khu vực hỗ trợ học vụ trong cùng một màn hình. Thanh bên chứa các liên kết tra cứu nhanh, trong khi khu vực chính hiển thị thông báo, lịch học và các thông tin liên quan đến tài khoản. Thiết kế này tạo một điểm vào thống nhất cho cả dữ liệu tĩnh lẫn dữ liệu cần tra cứu trực tiếp: các câu hỏi như kết quả học tập, lịch học, học phí hay lịch thi không cần chuyển sang trang mới mà được xử lý ngay trong cửa sổ chatbot, trả về từ đúng dữ liệu đang có trong cơ sở dữ liệu.

[CHÈN HÌNH 3.6: Ảnh giao diện Dashboard Sinh viên]

c) Cửa sổ Chatbot

Chatbot được nhúng trong dashboard dưới dạng một cửa sổ nhỏ, gồm khu vực hiển thị hội thoại, ô nhập câu hỏi, nút gửi và các gợi ý câu hỏi thường gặp. Mỗi câu trả lời của bot đi kèm các thao tác nghe, sao chép, đánh giá hữu ích hoặc báo lỗi, và một mục tùy chọn để yêu cầu hỗ trợ thêm. Chức năng nghe câu trả lời được đặt ngay cạnh nội dung trả lời thay vì ở một màn hình riêng, giúp sinh viên không phải tìm kiếm; khi người dùng chọn nghe một câu trả lời khác, đoạn âm thanh đang phát trước đó được dừng lại để tránh chồng chéo.

[CHÈN HÌNH 3.7: Ảnh cửa sổ Chatbot với câu hỏi, câu trả lời và các nút thao tác]

d) Màn hình ticket hỗ trợ

Ticket được thiết kế như một luồng trao đổi hai chiều. Sinh viên có thể tạo ticket từ chatbot hoặc từ khu vực hỗ trợ riêng, xem lại nội dung trao đổi và gửi phản hồi khi ticket chưa đóng. Ở phía Admin, ticket được hiển thị theo từng trạng thái xử lý, cho phép Admin mở cửa sổ trao đổi, phản hồi và đóng ticket khi đã giải quyết xong.

[CHÈN HÌNH 3.8: Ảnh cửa sổ trao đổi ticket phía Sinh viên và phía Admin]

e) Admin Dashboard

Trang quản trị có thanh điều hướng với các khu vực Tổng quan, Hỗ trợ Ticket, Kiểm duyệt tri thức, Quản lý Sinh viên và Lịch sử Chat. Khu vực Kiểm duyệt tri thức cho phép Admin thêm nguồn tài liệu, thêm hoặc chỉnh sửa nội dung tri thức, thay đổi trạng thái kiểm duyệt. Khu vực Lịch sử Chat cho phép Admin xem lại các phiên hội thoại, trạng thái câu trả lời, đánh giá của

sinh viên và ticket liên quan, phục vụ việc kiểm tra chất lượng phản hồi của bot theo thời gian. Cách tổ chức này nối trực tiếp dữ liệu vận hành với kho tri thức, thay vì tách việc xử lý phản hồi sang một công cụ nằm ngoài hệ thống.

[CHÈN HÌNH 3.9: Ảnh Admin Dashboard tại khu vực Tổng quan]

[CHÈN HÌNH 3.10: Ảnh khu vực Kiểm duyệt tri thức]

f) Quy tắc trình bày và khả năng sử dụng

Các thao tác đánh giá hoặc yêu cầu hỗ trợ được đặt ngay gần câu trả lời hoặc ticket tương ứng, nhằm giảm số bước thao tác khi người dùng muốn phản hồi. Đối với nội dung do người dùng nhập hoặc nội dung lấy từ cơ sở dữ liệu, hệ thống thực hiện xử lý để tránh hiển thị sai định dạng hoặc để lọt mã độc khi đưa dữ liệu ra giao diện, cả ở phía PHP lẫn phía JavaScript. Do phần giao diện gắn liền với môi trường chạy PHP và MySQL của nhóm, các ảnh chụp màn hình chính thức chỉ được đưa vào báo cáo sau khi nhóm kiểm tra lại phiên bản giao diện, tài khoản mẫu và dữ liệu hiển thị nhằm bảo đảm đúng với những gì hệ thống đang vận hành.

Như vậy, Chương 3 đã trình bày quá trình chuyển các mô hình khái niệm đã được duyệt ở Chương 2 thành cấu trúc cơ sở dữ liệu, các endpoint backend và luồng giao diện có thể vận hành trên môi trường thực tế. Những điều chỉnh như mở rộng schema để phục vụ cơ chế truy xuất tri thức và ticket, hay thay đổi cơ chế xác thực từ JWT sang PHP Session, đều là các quyết định phát sinh trong giai đoạn triển khai nhằm bảo đảm hệ thống vận hành thống nhất trên nền PHP, MySQL và service đọc văn bản hiện có, không làm thay đổi mục tiêu và phạm vi đã xác định từ Chương 1. Các kết quả kiểm thử, số liệu hiệu năng và đánh giá bảo mật sẽ được trình bày ở Chương 5 sau khi nhóm có dữ liệu kiểm thử thực tế trên hệ thống đã triển khai.